Supporting Information for Visualizing Chemical Principles Using Interactive Python Modules

Lindsay Clos†, Katherine Smith†, Matthew D. Hanson*

Department of Chemistry, Le Moyne College, Syracuse, NY 13214, USA
*Email: hansoma@lemoyne.edu

Contents

1 About This Guide **S2**

2 Python and Package Installation **S3**
2.1 Quick Installation Guide S3
2.2 Additional Installation Explanations S3

3 Running the Programs **S6**

4 Program Features, Conventions, and Select Data **S7**
4.1 Born-Haber_GUI.py S7
4.2 Heating_Curve_GUI.py S9
4.3 Kinetic_Order_Plots_GUI.py S10
4.4 Activation Energy Plots S11
4.5 Gibbs Free Energy and Equilibrium Plots S12
4.6 Acid-Base Titration Plots S13
4.7 Ideal and Van Der Waals Gases S15
4.8 Maxwell-Boltzmann Distributions S17
4.9 Morse Potential Curves S18

5 Derivation of the Speciation Equations **S19**
5.1 Monoprotic Acid Speciation S19
5.2 Diprotic Acid Speciation S19
5.3 Triprotic Acid Speciation S20

1 About This Guide

This document includes instructions for installing and using the GUI-based Python modules outlined in the main article. Each of these modules is a standalone Python script that needs no separate installation once the necessary Python software has been installed. These modules can be run either using an interpreter like the IDLE interpreter that comes with Python or with a command line command. The features of each of these modules and the conventions for their proper use are also outlined in this document. For convenience, a set of parameters for some common molecules or links to easily accessible data are given to aid instructors with using these tools in the classroom or for generating problems, where appropriate. Note that the codes are designed to be robust so that users inputting physically unreasonable values will result in guidance messages rather than an error whenever possible.

2 Python and Package Installation

2.1 Quick Installation Guide

The following steps should be followed to install the necessary packages locally on a computer:

1. Download and install Python 3. **Windows users should check the box that installs Python3 on the user's PATH during installation!**

2. Install and/or open a terminal application and run the following command to install the necessary Python packages and their dependencies,

   ```
   python3 -m pip install numpy scipy matplotlib
   ```

   You may be prompted to confirm the downloads of various packages by typing "Y" and Enter/return into the terminal. If this does not run properly, try removing the "3" from the end of the python3 command. If one or more packages fails to install properly it is worth trying to install the package individually using the same installation command with only the offending package(s).

All modules should now be usable on your computer either through an interpreter like IDLE or via command line.

2.2 Additional Installation Explanations

Python is an extremely popular and relatively user-friendly coding language. The need for installation instructions comes from the various Python packages used in the Python scripts and their large number of dependencies. For Windows users, be sure to choose the option to include Python on the user's PATH when first installing it or else the executable command for Python in the terminal will not actually work! The PATH environment variable is where the computer looks to find executable files for various programs. You can also manually modify the PATH on Windows by searching for the environment variables and adding a new PATH variable corresponding to the locations of executable files on your computer. This can be used if Python is accidentally installed without adding the executable to the PATH, but will not be covered here.

The reason we need to install additional modules is that Python does not have all available features activated unless they are requested via the import command. Some packages with niche technical uses are not installed by default, which necessitates obtaining them from an external source. This allows programs to use fewer computer resources that would be wasted processing unnecessary computer code and saves hard drive space for those that do not need certain packages. In each program, there is a block of code that imports the necessary Python packages at the start of the module, as shown in the code sample in Figure S1.

To install the necessary packages on either Windows or MacOS computers, just run the following command in a terminal application once Python has been installed,

```
python3 -m pip install numpy scipy matplotlib
```

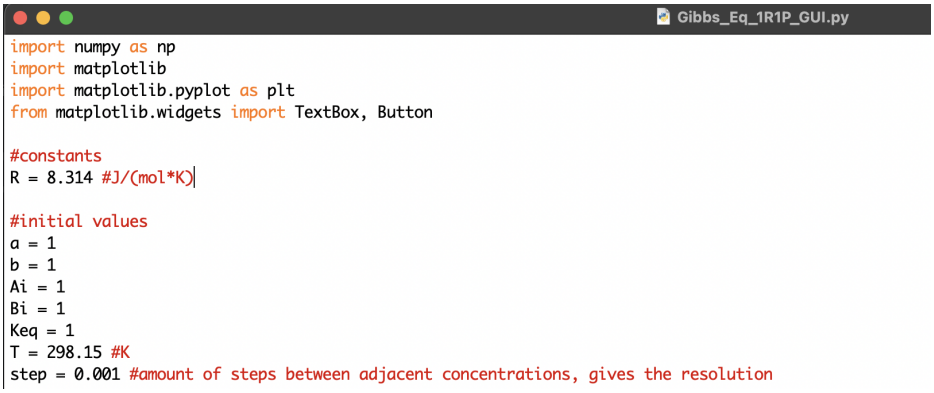


Figure S1: Example of Python code that imports packages and then defines variables.

On a Mac terminal application the terminal may look like the example in Figure S2. In this example, the command `pwd` prints the path to the current directory that the terminal is accessing. To find the terminal application, on MacOS simply open a Finder window and navigate to Applications/Utilities/Terminal. It is recommended to make the Terminal application a shortcut on the user's toolbar for ease of use. On Windows, simply type "PowerShell" into the search bar near the Start menu and select the Windows PowerShell application. If PowerShell is not already installed, it can be downloaded for free from the Microsoft store. Once again, it is recommended that the user pin a terminal application to their toolbar for ease of access later. No knowledge of operating the terminal is required to utilize these modules other than following the commands explicitly laid out in these instructions.

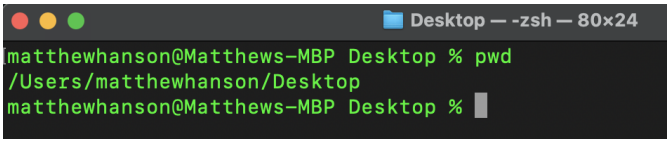


Figure S2: Example of terminal code that prints the current working directory.

The "pip install" command is a built-in command included with Python that is used to install additional packages to the Python distribution. The pip command for Python3 is also frequently used in the form,

```
pip3 install numpy
```

but the version of the command,

```
python3 -m pip install
```

ensures that executable files are installed on the user's PATH for Windows machines. Note that some installations of Python will only recognize the execution command "python" in a terminal, so the above installation command can be modified by removing the "3" from the end of the python3 command. Note that the above commands can be run for individual packages (e.g. "pip3 install numpy") or for some small subset of the packages (e.g. "python3 -m pip install numpy scipy"), if desired, though it is easiest to install many at once. Note

that when running pip on its own, the command pip3 must be run for installation of packages into Python3. Use of "pip install" rather than "pip3 install" will attempt to install packages for Python2 for MacOS users, which has now been deprecated despite being installed for MacOS by default.

Some of the necessary packages may already be installed depending on the user's Python distribution. For instance, the numpy and scipy modules are typically installed in Anaconda distributions of Python. Attempting to install packages that are already installed will not cause problems, but the user will see a large number of messages in the terminal indicating that the package dependencies are already satisfied.

Package managers like Anaconda and Mamba, which can be downloaded by following the relevant hyperlinks, are also capable of running Python in a virtual environment. This circumvents the need for local installations of all the requisite packages. This can be important for particular computer environments. To create a virtual environment called "pymods" with the necessary Python packages in Mamba, use the following commands in a terminal,

```
mamba create -n pymods python numpy scipy matplotlib
mamba activate pymods
python Module_Name.py
```

where `Module_Name` should be replaced with the desired module name. When you are finished with the program, the following terminal commands can be used to terminate the virtual environment,

```
mamba deactivate
mamba remove -n pymods --all
```

The same process of creating a virtual environment can be achieved in Anaconda by using the terminal commands,

```
conda config --add channels conda-forge
conda config --set channel_priority strict
conda create -n pymods python numpy scipy matplotlib
conda activate pymods
python Module_Name.py
```

where again `Module_Name` must be replaced with the name of the desired module. The virtual environment can be deactivated using the terminal commands,

```
conda deactivate
conda remove -n pymods --all
```

3 Running the Programs

The module file can be opened in an editor like the IDLE interpreter that comes with Python and run through the interpreter's shell. Typically, double clicking opens Python files in IDLE by default. The module can be run in IDLE by pressing the F5 key or by going through the Run menu. Note that because of imperfections in the Python launcher the widgets can sometimes become "stuck." The best solution is to simply close the window and re-run the code after a few seconds.

Alternatively, users can also run the modules through the command line. To run the Python script files when the `Chemistry_Python_Modules` folder is placed on the user's Desktop, the following steps can be used:

1. Open the terminal application and type the following commands sequentially,

   ```
   cd Desktop/Chemistry_Python_Modules
   python3 Module_Name.py
   ```

   where the `Module_Name` is the name of the desired module. If the latter fails, try removing the "3" from the python3 command.

2. Utilize the interactive GUI elements to update or create plots as desired. The window for the program can be closed to end the program.

4 Program Features, Conventions, and Select Data

This section will outline the features of the various Python modules by topic. Modules that are functionally similar will be grouped together. These topics are in no particular order due to their variety. The chemical conventions of each program will also be stated as needed.

4.1 Born-Haber_GUI.py

The first module we will introduce is one of the most complex. The Born-Haber_GUI.py module allows the automatic generation of plots that show the Born-Haber cycle for the formation of an ionic solid of the form M_aN_b, where M is a metal atom and N is a non-metal atom, from its constituent elements in their natural state, as shown in Figure S3. The

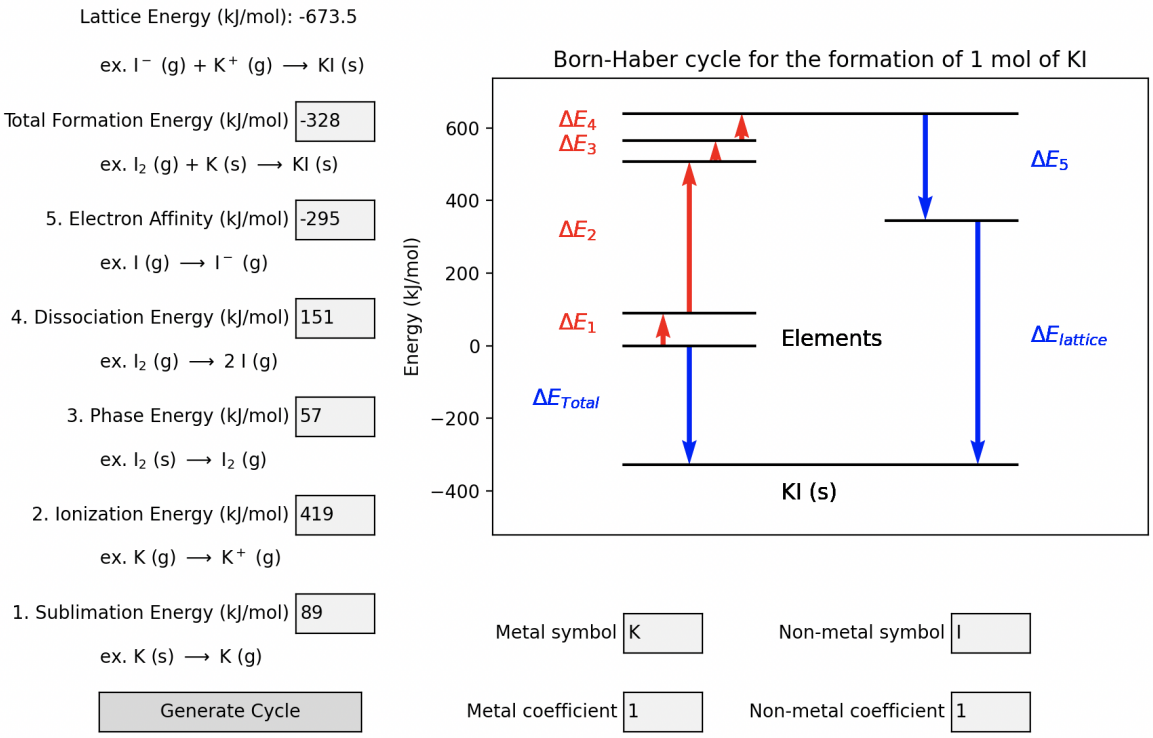


Figure S3: Example of the Born-Haber_GUI.py module.

program's primary usage is showing how the lattice energy varies for various ionic solids; the lattice energy displayed at the top of the user inputs automatically adjusts whenever the "Generate Cycle" button is pressed. The program also generates a graphic with the full Born-Haber cycle, including arrows that are generally colored based on the sign of the energy involved in that step of the process. The program takes in the energy (or enthalpy–the plot will just say energy on the y axis and these values are usually almost the same) required for each of several reaction steps:

1. The sublimation (or vaporization, in the unique case of mercury) energy of the metal element. This is ΔE_1 on the graph.

2. The total ionization energy of the metal atom. For cations of higher charge, all of the relevant ionization energies must be added together. Ionization energy data are readily available on the excellent Ptable.com. This is ΔE_2 on the graph.

3. The energy of any phase transitions necessary to bring the non-metal element in their natural state into the gas phase. If the element is already naturally in the gas phase, this value can be left as 0 and the corresponding step will be omitted from the produced graphic. This would be ΔE_3 if it were shown on the example graph.

4. The dissociation energy of a diatomic non-metal element, if applicable. If this energy is left as 0 due to the non-metal element being monoatomic in its standard state, the relevant step in the cycle will also be omitted. Note that the reaction stoichiometry assumes that one mole of a diatomic molecule is dissociated, producing a pair of atoms. Thus, only half of this energy is used in the calculation of the lattice energy! This is ΔE_4 on the graph.

5. The electron affinity of the non-metal atom. These are typically negative by convention in this program. Electron affinity data for adding a single electron to an atom is readily available on Ptable.com, though this data is listed with typically positive values by convention on the PTable site. This is ΔE_5 on the graph.

6. The overall energy of formation for the ionic solid from its constituent elements. This is ΔE_{Total} on the graph.

The program also takes into account the stoichiometry of the solid M_aN_b with the coefficients a and b corresponding to the metal atom M and non-metal atom N when determining the lattice energy with the formula,

$$\Delta E_{lattice} = \Delta E_{Total} - a\Delta E_1 - a\Delta E_2 - b\Delta E_3 - \frac{b}{2}\Delta E_4 - b\Delta E_5$$

This allows for the quantitative analysis of solids like alkali metal oxides or alkaline earth metal halides with no issues. The default values that the program uses are for the formation of the solid KI. Some values for a small selection of solids with various cations and anions are given in Table S1. In this table, each energy corresponds to the appropriate element or ion necessary to make the solid and the ionization and electron affinity data has been summed for the necessary charges. The lattice energy $\Delta E_{lattice}$ calculated from the program is given in the final column, all of which agree reasonably well with accepted values.

Table S1: Constant values for the Born-Haber cycle in kJ/mol

Solid	ΔE_1	ΔE_2	ΔE_3	ΔE_4	ΔE_5	ΔE_{Total}	$\Delta E_{lattice}$
LiF	161	520	0	155	-328	-617	-1047.5
LiCl	161	520	0	243	-349	-409	-862.5
NaCl	107	496	0	243	-349	-411	-786.5
KCl	89	419	0	243	-349	-438	-718.5
KI	89	419	57	151	-295	-328	-673.5
Na_2O	107	496	0	495	603	-416	-2472.5
CaO	178	1735	0	495	603	-603	-3366.5
$CaBr_2$	178	1735	31	193	-325	-675	-2193

4.2 Heating_Curve_GUI.py

The Heating_Curve_GUI.py module can be used to generate heating curves for pure substances. This module takes into account the various heat capacities of each phase of the substance, the phase transition temperatures, the molar enthalpies of each of these phase transitions, the amount of substance, its molar mass, and allows the user to specify the temperature range of interest. This is shown in the example in Figure S4. The program will

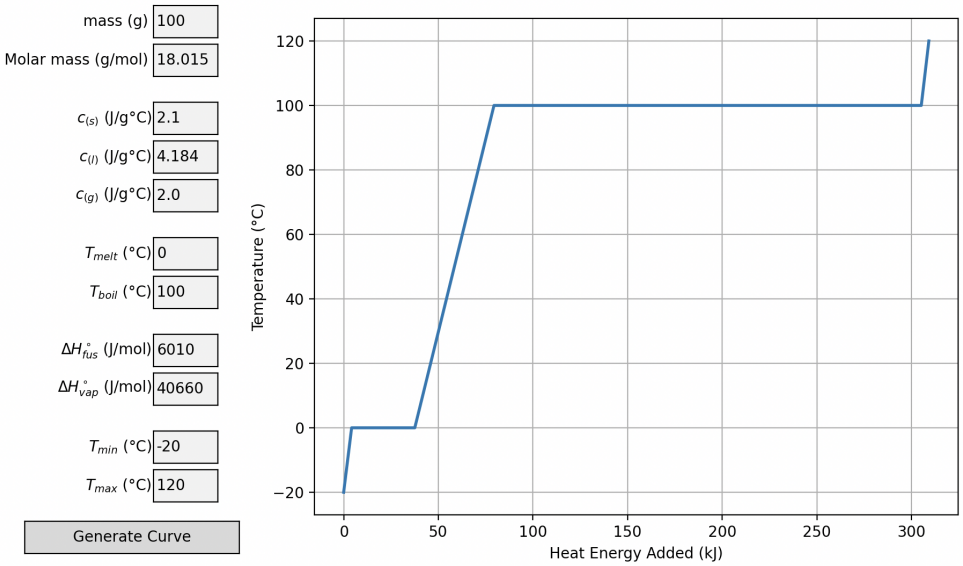


Figure S4: Example of a heating curve for water.

also read off precise values of the heat involved in the temperature changing regions and any phase transitions, as shown in Figure S5. The isobaric specific heat capacities (in J/g°C), melting and boiling points (°C), standard enthalpy of fusion and vaporization (J/mol), and molar masses of a few select substances are given in Table S2 for convenience. Note that graphite does not melt; it directly sublimates to the gas phase. Thus, we listed the heat capacity of the liquid phase as 0, the enthalpy of fusion as 0, and the melting point as the same as the boiling point so the calculations will come out correctly.

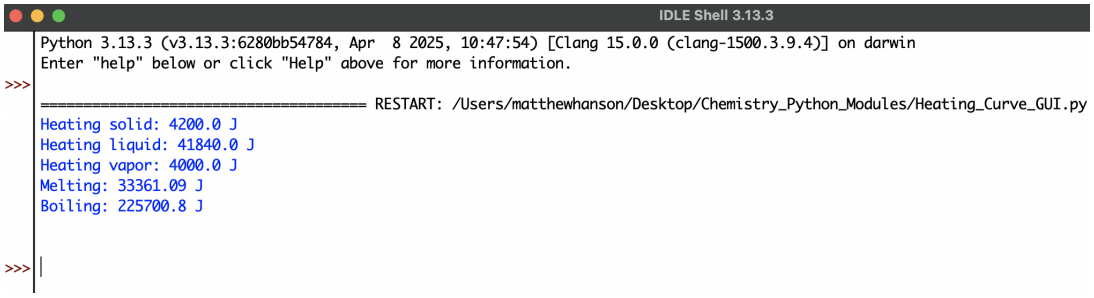


Figure S5: Example of data printed to the consolve by Heating_Curve_GUI.py.

Table S2: Constants relevant for heating curves

Substance	Molar mass	$c_{(s)}$	$c_{(l)}$	$c_{(g)}$	T_{melt}	T_{boil}	ΔH°_{fus}	ΔH°_{vap}
H_2O	18.015	2.1	4.184	2.0	0	100	6010	40660
NH_3	17.03	2.1	4.7	2.16	-77.7	-33.3	5650	23500
Cu	63.546	0.385	0.572	0.327	1085	2562	205	300300
CH_3CH_2OH	46.069	0.97	2.50	1.41	-114.1	78.4	46000	42300
C (graphite)	12.011	0.72	0*	0.327	3650*	3650	0*	717000

4.3 Kinetic_Order_Plots_GUI.py

This module allows for the generation of plots that can be used to test students' understanding of simple integrated rate laws and how visual data can be used to determine the order of reaction quite easily. As shown in Figure S6, the plots of [A], ln([A]), and 1/[A] vs. time can be used to determine the order of reaction since these would correspond to linear plots for 0th, 1st, or 2nd order single-component reactions, respectively. Changing the fraction of the initial concentration remaining can help to illustrate how difficult it can be to distinguish between the various reaction orders unless a reaction is monitored for a sufficiently long time.

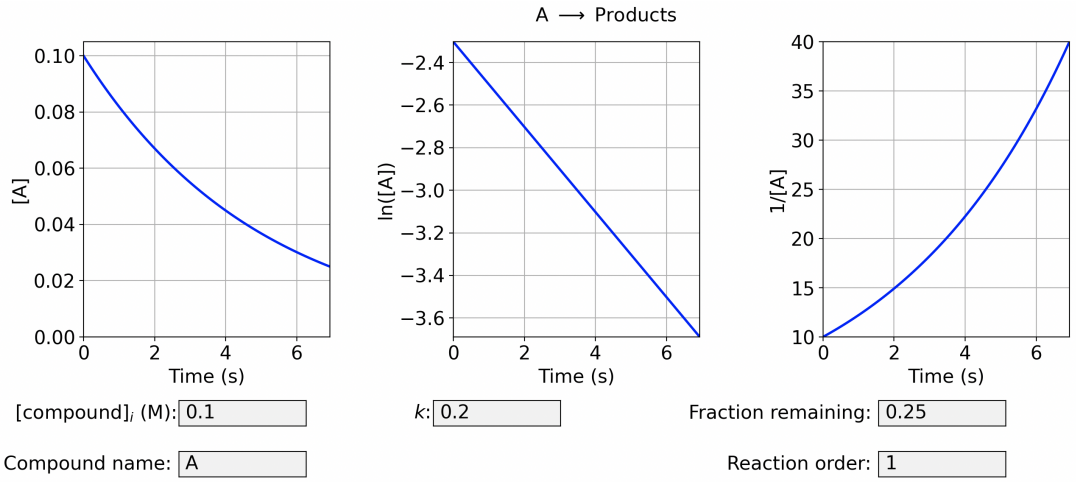


Figure S6: Examples of plots that can be compared to the linearized integrated rate laws to determine reaction order.

The boxes also allow for putting a name of the reactant, with LaTeX conventions used for subscripts and superscripts (i.e. CO$_3^{2-}$ gives CO_3^{2-}). The user can also specify the initial concentration and rate constant. The units of the rate constant are omitted because they vary with the order of reaction. This should be emphasized by instructors by practicing the determination of the rate constant units from reaction order alone.

4.4 Activation Energy Plots

The Activation_Energy_GUI.py module allows for the generation of graphs that qualitatively illustrate the concept of activation energy for reactions that can be endoergic or exoergic with or without the presence of reaction intermediates. Because the energies involves in reaction mechanisms are generally quite complex, this is done qualitatively by stitching together Gaussian (bell curve) functions with heights controlled by the relevant activation energies, reactant and product energies, and intermediate energies (where appropriate). The width of the Gaussians is controlled by a parameter w. This is shown in the examples of single and two step exoergic processes without catalysts in Figure S7. This module utilizes

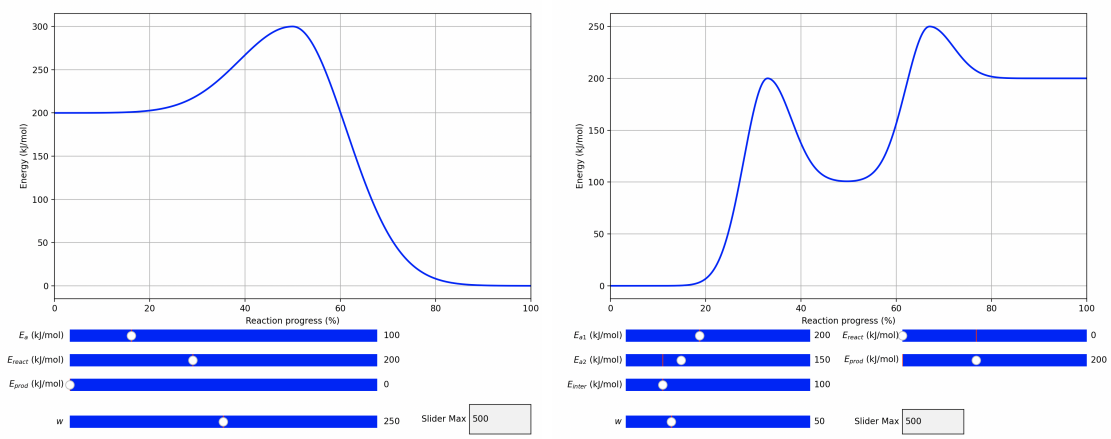


Figure S7: Examples of single and two step uncatalyzed exoergic (left) and endoergic (right) reaction energy profiles.

sliders to allow users to dynamically update the graphs in real time. The sliders have built-in safety checks to ensure that the values do not result in physically nonsensical situations (i.e. the activation energy being insufficient to reach the next step of the reaction). The sliders are limited to the value set in the Slider Max box, which can be adjusted to give a broader or smaller range as needed. The "catalyst" options can be used to illustrate how a catalyst might change the mechanism of a reaction by lowering the activation energy and potentially adding additional steps. For simplicity, only a single-step uncatalyzed reaction is shown in this case, as shown in Figure S8 for an endothermic process. Only the parameter explicitly labeled as "uncatalyzed" directly affects this process. All other parameters refer to the catalyzed reaction profile, as this is the focus of these sub-modules.

The activation energy of the uncatalyzed process will automatically adjust if necessary to ensure that the catalyzed process is at least as fast as the uncatalyzed process.

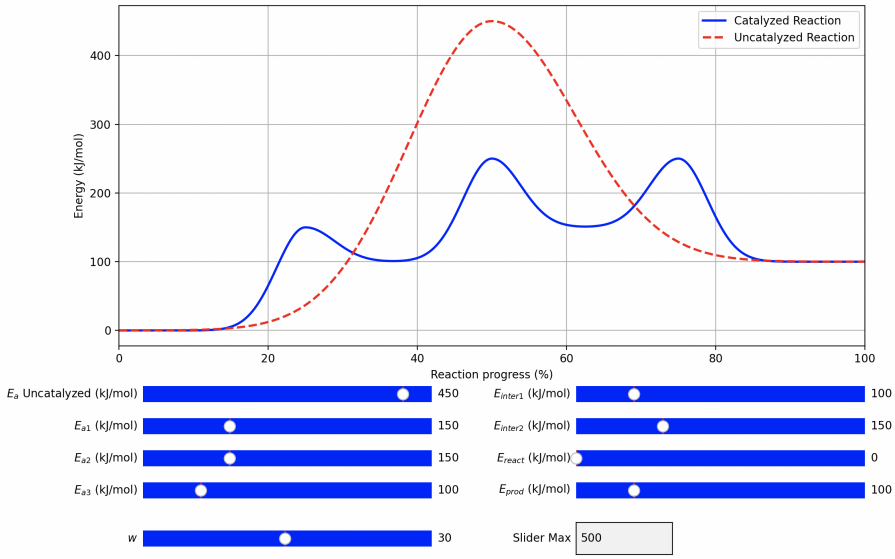


Figure S8: Example of two step catalyzed reaction energy profile.

4.5 Gibbs Free Energy and Equilibrium Plots

The Python module Gibbs_Free_Energy_GUI.py utilizes the connection between thermochemistry and equilibrium in the familiar equations,

$$\Delta G^{\circ}_{rxn} = -RT\ln(K_{eq})$$
$$\Delta G_{rxn} = \Delta G^{\circ}_{rxn} + RT\ln(Q)$$

to illustrate how different values of the initial concentrations of species with various stoichiometric coefficients give rise to different equilibrium concentrations of these species at a certain temperature. The different options differ only in the number of reactants and products in the reaction shown above the graphs, which allows instructors flexibility in their choices of examples. The example in S9 is for a two reactant and two product reaction. Each module shows how equilibrium can be considered established in one of three ways:

1. The reaction quotient Q is the same as the equilibrium constant K_{eq}.
2. The $\Delta G_{rxn} = 0$.
3. The relative value of G for all of the reactants and products present has reached a minimum.

These modules allow both the qualitative and quantitative presentation of complicated relationships between thermodynamic values and the ultimate equilibrium that is established. For ease of use, it is recommended that equilibrium constants not be made too large or small.

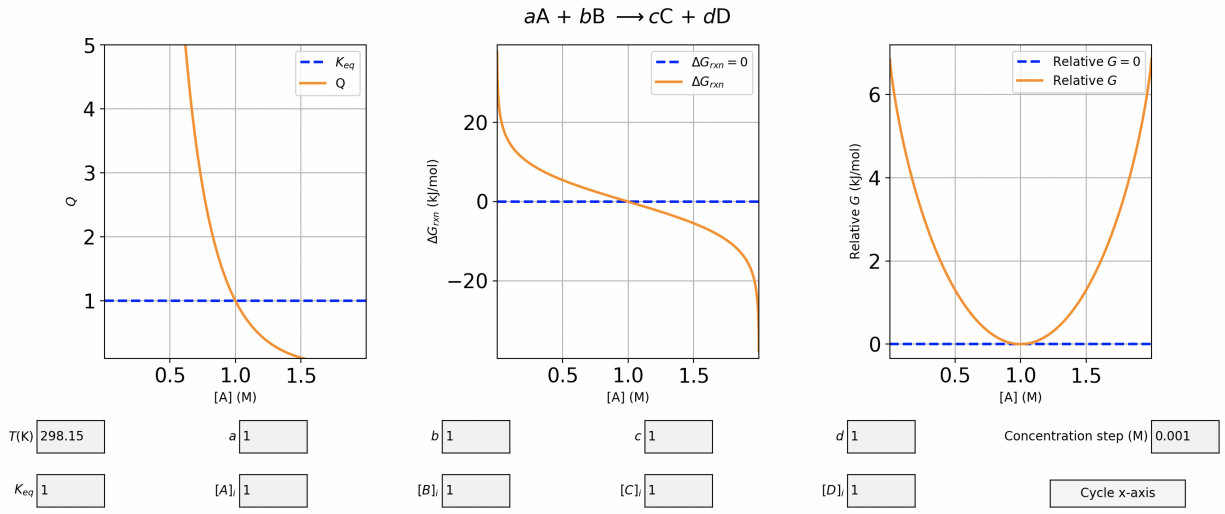


Figure S9: Example of the Gibbs free energy and reaction quotient plots for a particular two reactant and two product reaction.

4.6 Acid-Base Titration Plots

The Titration_Curves_GUI.py Python module can be used to generate the titration curves of mono- di- and triprotic acids with a strong base. The first three options merely create a titration curve with the user controlling the pK_a value(s) via a slider and with the concentration of the acid and base solutions and volume of the original acid sample controlled by text boxes, as shown in the example in Figure S10.

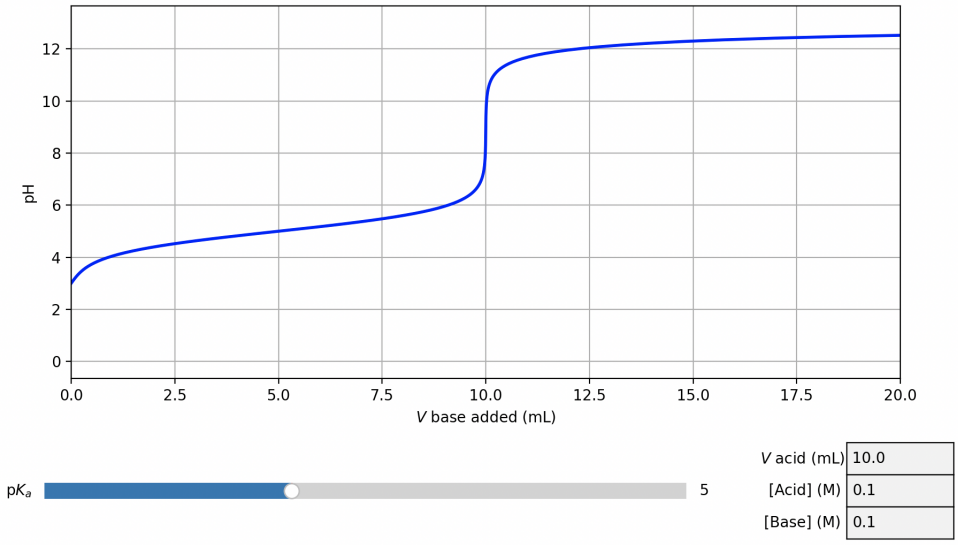


Figure S10: Example of a monoprotic titration curve generated with the Python module.

The modules that include speciation in their names additionally create a second subplot that shows the concentrations of all forms of the acid species in solution at every point along the titration curve, as shown in Figure S11. The equations used for the concentration of

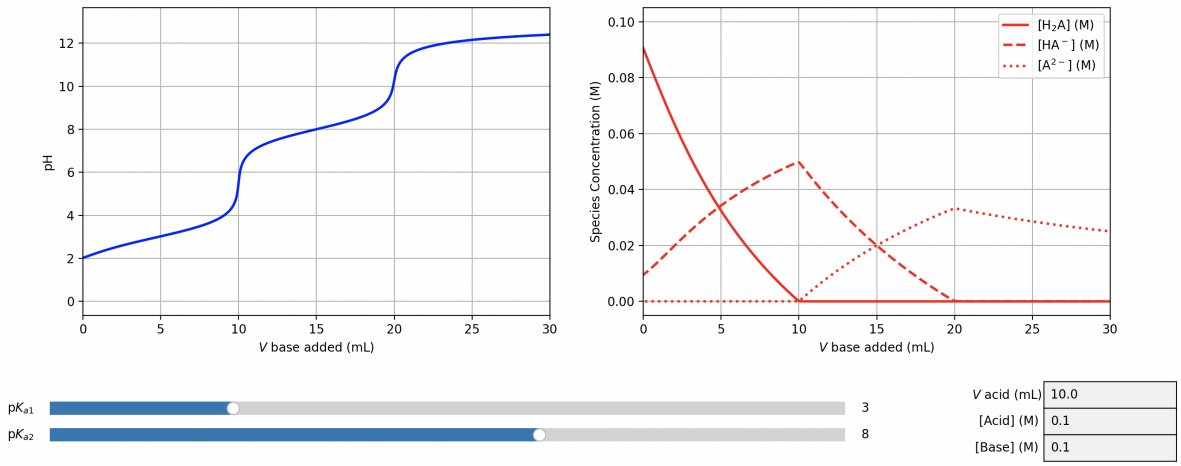


Figure S11: Example of the speciation curves (right) for a diprotic acid titration curve (left).

each species are derived using the conservation of the total amount of the acid species in solution and the Henderson-Hasselbalch equation, as shown in the last section. Note that the dilution of the acid is taken into account in the calculation of the concentrations.

The derivative options instead create a second subplot that shows the slope of the titration curve at every point along the curve, as shown in Figure S12. The derivative plot is useful

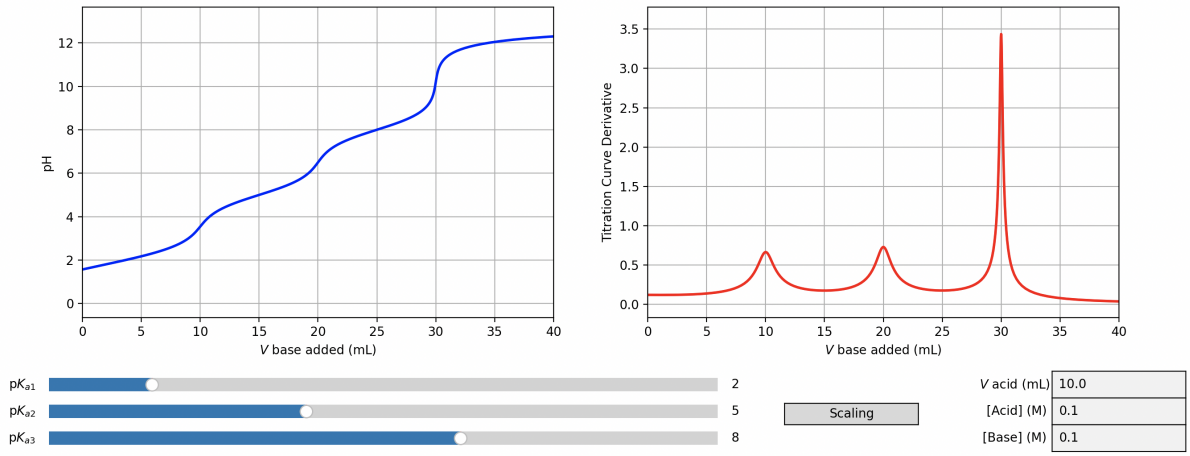


Figure S12: Example of the derivative plot (right) for a diprotic acid titration curve (left).

for spotting subtle endpoints of polyprotic acids. The scaling button switches the graph between scaling to the highest peak (default) or the lowest peak, which can be useful for identifying difficult endpoints.

4.7 Ideal and Van Der Waals Gases

The Ideal_VDW_Gases_GUI.py module creates a 3-D surface plot that displays the relationship between V, T, and P in the ideal gas law for a fixed amount $n = 1$ mol of gas according to the ideal gas law. It also uses sliders to allow the user to move a point around on this surface to see how the pressure varies with changing volume and temperature more easily. The moles of gas can be adjusted using the dedicated text box. Because 3-D plots can be somewhat awkward to maneuver in Python, dedicated buttons that orient the viewing window along the x, y, and z axes are added at the top of the window. The pressure at the point that moves about the surface is displayed on the bottom right, as shown in Figure S13. The Ideal_VDW_Gases_GUI.py also module creates a 3-D surface plot of how the pressure

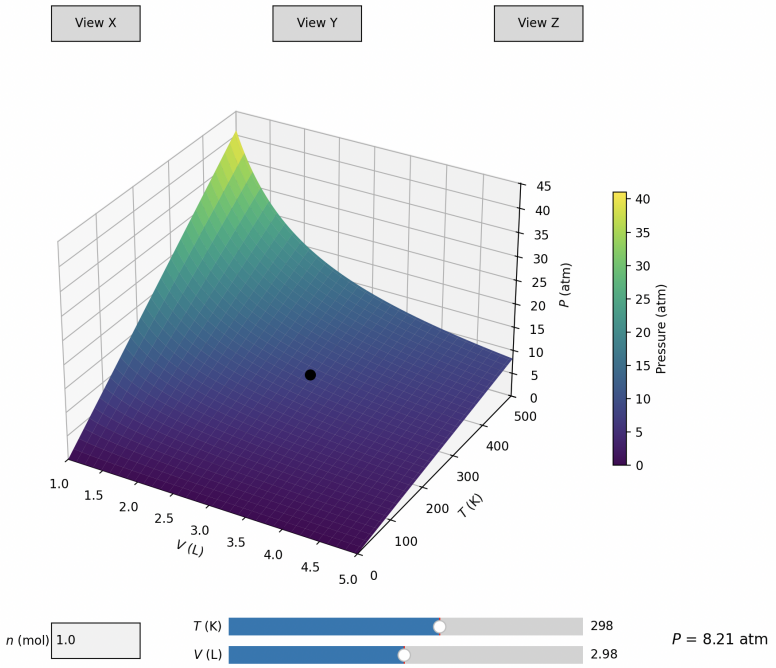


Figure S13: Example of the surface plot generated for an ideal gas.

of $n = 1$ mol of a Van Der Waals (VDW) gas varies with both the temperature and volume. The plots produced by this module often look highly similar to the ideal gas plot for typical values of the Van Der Waals parameters a (the pressure correcting term) and b (the volume correcting term). To facilitate identifying the differences, an additional button is added on the top right that switches to showing the deviation of the VDW gas pressure from the ideal gas. A negative deviation means that the VDW gas has a lower pressure. The same functionality of moving a point about on the surface exists in both modes. Two new boxes are added for changing the parameters a and b. Both modes are shown in Figure S14. The Van Der Waals parameters for a few select substances are given in Table S3 for convenience. Note that negative gas pressures are a specialized artifact of the VDW equation.

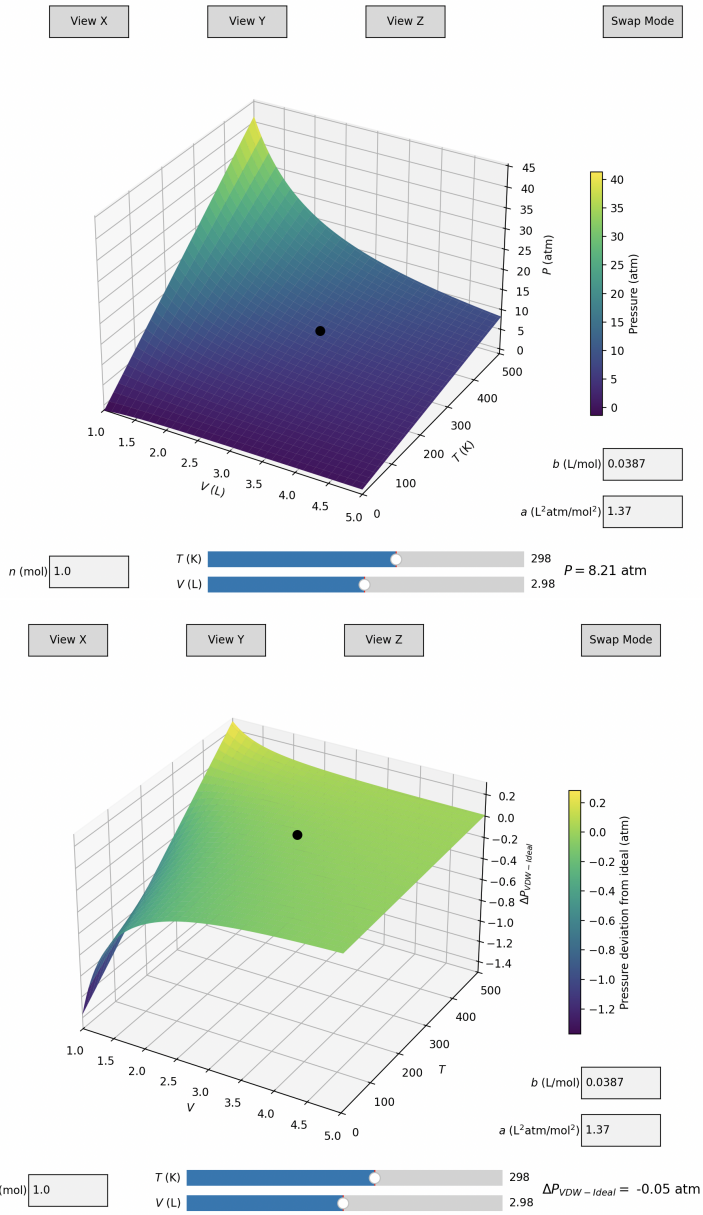


Figure S14: (Top) Example of the surface plot generated for a Van Der Waals gas and (bottom) the deviation of the gas pressure from the corresponding ideal gas.

Table S3: Example Van Der Waals parameters for simple gases.

Gas	a (L^2atm/mol^2)	b (L/mol)
Ne	0.214	0.0171
N_2	1.370	0.0387
O_2	1.382	0.0319
CO_2	3.60	0.0427
N_2H_4	8.46	0.0462

4.8 Maxwell-Boltzmann Distributions

The Maxwell-Boltzmann_GUI.py module allows the user to create Maxwell-Boltzmann distributions of the translational speeds of gas molecules based on the molar mass of the gas and the temperature. There is also a button to change the axes from constantly rescaling to maintain the aspect ratio or fixing the current axes and just updating the curve. This allows for more dynamic control of the visuals and different emphases for teaching. Examples of the outputs of these codes are shown in Figure S15.

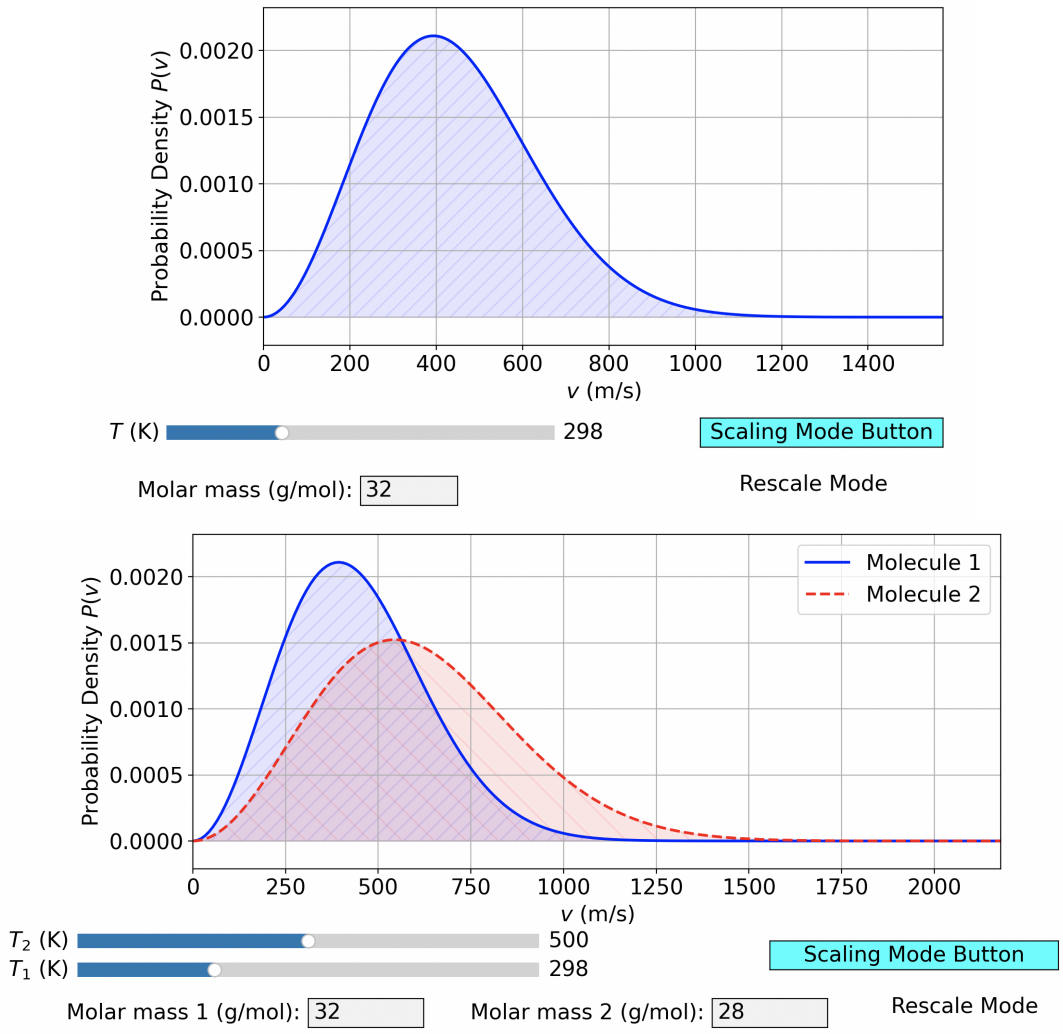


Figure S15: (Top) Maxwell-Boltzmann distribution for a single gas and (Bottom) Maxwell-Boltzmann distributions for a pair of gases.

4.9 Morse Potential Curves

The Morse_Potential_GUI.py module allows the user to create Morse potential curves that show bond dissociation energies (D_e), stable equilibrium bond lengths (r_e), and slight differences in potential shape (determined by a) for various molecules, with the option to compare a pair of bonding energy curves simultaneously. Examples are shown in Figure S16. Parameters for select diatomic molecules are given in Table S4 for convenience.

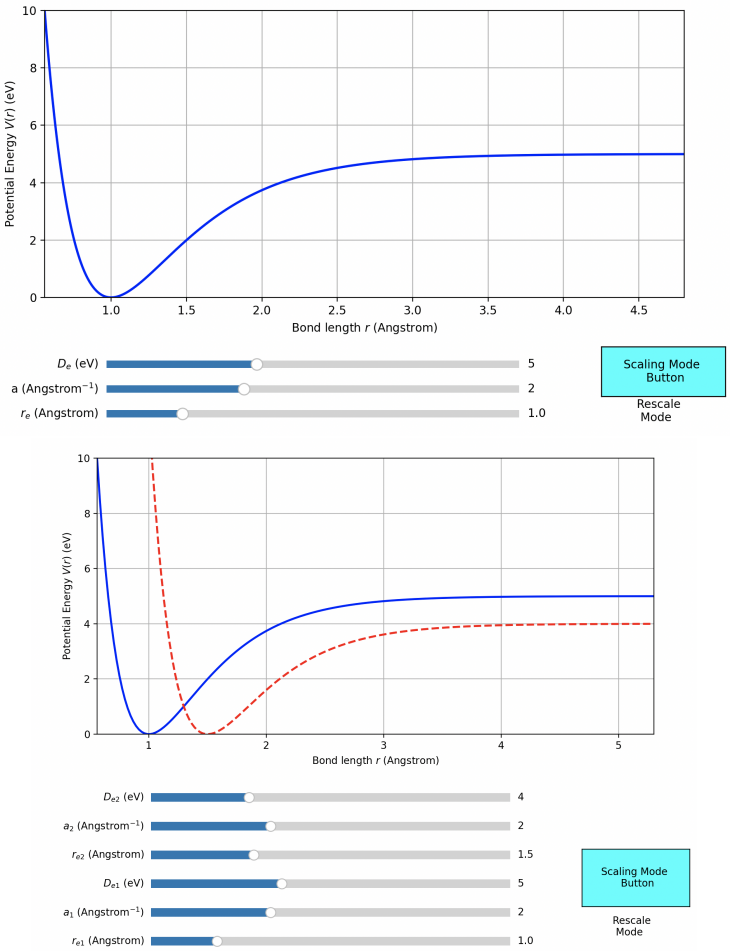


Figure S16: (Top) Morse potential curve for a single bond and (Bottom) Morse potential curves for a pair of different bonds.

Table S4: Table of select Morse parameters for various diatomic molecules' bonds.

Molecule	D_e (eV)	a (Å^{-1})	r_e (Å)
I_2	1.54	1.80	2.666
H_2	4.75	1.94	0.741
HCl	4.43	1.87	1.274
NO	6.50	2.50	1.151
CO	11.09	2.63	1.128

5 Derivation of the Speciation Equations

5.1 Monoprotic Acid Speciation

For a monoprotic acid with protonation states represented by HA and A^-, we can write the conservation of the total concentration of all forms of the acid as,

$$[\mathrm{HA}] + \left[\mathrm{A}^-\right] = \text{total } [\mathrm{A}]$$

The Henderson-Hasselbalch equation gives,

$$\begin{aligned}
\mathrm{pH} &= \mathrm{p}K_a + \log_{10}\left(\frac{\left[\mathrm{A}^-\right]}{[\mathrm{HA}]}\right) \\
\implies \left[\mathrm{A}^-\right] &= [\mathrm{HA}] \cdot 10^{\mathrm{pH}-\mathrm{p}K_a} \\
\implies \text{total } [\mathrm{A}] &= [\mathrm{HA}]\left(1 + 10^{\mathrm{pH}-\mathrm{p}K_a}\right) \\
\implies [\mathrm{HA}] &= (\text{total } [\mathrm{A}])\left(1 + 10^{\mathrm{pH}-\mathrm{p}K_a}\right)^{-1}
\end{aligned}$$

Using the conservation of concentration gives the working equations,

$$\begin{aligned}
[\mathrm{HA}] &= (\text{total } [\mathrm{A}])\left(1 + 10^{\mathrm{pH}-\mathrm{p}K_a}\right)^{-1} \\
\left[\mathrm{A}^-\right] &= \text{total } [\mathrm{A}] - [\mathrm{HA}]
\end{aligned}$$

The dilution can be taken into account by modifying total [A] with the dilution factor.

5.2 Diprotic Acid Speciation

As with the monoprotic acid speciation, we can write the conservation of the concentrations of all species for a diprotic acid H_2A as,

$$[\mathrm{H_2A}] + \left[\mathrm{HA}^-\right] + \left[\mathrm{A}^{2-}\right] = \text{total } [\mathrm{A}]$$

The Henderson-Hasselbalch equation for each $\mathrm{p}K_a$ gives,

$$\begin{aligned}
\mathrm{pH} &= \mathrm{p}K_{a1} + \log_{10}\left(\frac{\left[\mathrm{HA}^-\right]}{[\mathrm{H_2A}]}\right) \\
\mathrm{pH} &= \mathrm{p}K_{a2} + \log_{10}\left(\frac{\left[\mathrm{A}^{2-}\right]}{\left[\mathrm{HA}^-\right]}\right) \\
\implies \left[\mathrm{HA}^-\right] &= [\mathrm{H_2A}] \cdot 10^{\mathrm{pH}-\mathrm{p}K_{a1}} \\
\implies \left[\mathrm{A}^{2-}\right] &= \left[\mathrm{HA}^-\right] \cdot 10^{\mathrm{pH}-\mathrm{p}K_{a2}} \\
&= [\mathrm{H_2A}] \cdot 10^{2\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}} \\
\implies \text{total } [\mathrm{A}] &= [\mathrm{H_2A}]\left(1 + 10^{\mathrm{pH}-\mathrm{p}K_{a1}} + 10^{2\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}}\right) \\
\implies [\mathrm{H_2A}] &= (\text{total } [\mathrm{A}])\left(1 + 10^{\mathrm{pH}-\mathrm{p}K_{a1}} + 10^{2\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}}\right)^{-1}
\end{aligned}$$

Using the conservation of the total concentration gives the working equations,

$$\begin{aligned}
[\mathrm{H_2A}] &= (\text{total } [\mathrm{A}]) \left(1 + 10^{\mathrm{pH}-\mathrm{p}K_{a1}} + 10^{2\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}}\right)^{-1} \\
\left[\mathrm{HA^-}\right] &= [\mathrm{H_2A}] \cdot 10^{\mathrm{pH}-\mathrm{p}K_{a1}} \\
\left[\mathrm{A^{2-}}\right] &= \text{total } [\mathrm{A}] - [\mathrm{H_2A}] - \left[\mathrm{HA^-}\right]
\end{aligned}$$

where the dilution can be taken into account by modifying total [A] with the dilution factor.

5.3 Triprotic Acid Speciation

Fianlly, as with the monoprotic and diprotic acid speciation, we can write the conservation of the concentrations of all species for a triprotic acid $\mathrm{H_3A}$ as,

$$[\mathrm{H_3A}] + \left[\mathrm{H_2A^-}\right] + \left[\mathrm{HA^{2-}}\right] + \left[\mathrm{A^{3-}}\right] = \text{total } [\mathrm{A}]$$

The Henderson-Hasselbalch equation for each $\mathrm{p}K_a$ gives,

$$\begin{aligned}
\mathrm{pH} &= \mathrm{p}K_{a1} + \log_{10}\left(\frac{\left[\mathrm{H_2A^-}\right]}{[\mathrm{H_3A}]}\right) \\
\mathrm{pH} &= \mathrm{p}K_{a2} + \log_{10}\left(\frac{\left[\mathrm{HA^{2-}}\right]}{\left[\mathrm{H_2A^-}\right]}\right) \\
\mathrm{pH} &= \mathrm{p}K_{a3} + \log_{10}\left(\frac{\left[\mathrm{A^{3-}}\right]}{\left[\mathrm{HA^{2-}}\right]}\right) \\
\implies \left[\mathrm{H_2A^-}\right] &= [\mathrm{H_3A}] \cdot 10^{\mathrm{pH}-\mathrm{p}K_{a1}} \\
\implies \left[\mathrm{HA^{2-}}\right] &= \left[\mathrm{H_2A^-}\right] \cdot 10^{\mathrm{pH}-\mathrm{p}K_{a2}} \\
&= [\mathrm{H_3A}] \cdot 10^{2\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}} \\
\implies \left[\mathrm{A^{3-}}\right] &= \left[\mathrm{HA^{2-}}\right] \cdot 10^{\mathrm{pH}-\mathrm{p}K_{a3}} \\
&= [\mathrm{H_3A}] \cdot 10^{3\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}-\mathrm{p}K_{a3}} \\
\implies \text{total } [\mathrm{A}] &= [\mathrm{H_3A}] \left(1 + 10^{\mathrm{pH}-\mathrm{p}K_{a1}} + 10^{2\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}} + 10^{3\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}-\mathrm{p}K_{a3}}\right) \\
\implies [\mathrm{H_3A}] &= (\text{total } [\mathrm{A}]) \left(1 + 10^{\mathrm{pH}-\mathrm{p}K_{a1}} + 10^{2\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}} + 10^{3\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}-\mathrm{p}K_{a3}}\right)^{-1}
\end{aligned}$$

Using the conservation of the total concentration gives the working equations,

$$\begin{aligned}
[\mathrm{H_3A}] &= (\text{total } [\mathrm{A}]) \left(1 + 10^{\mathrm{pH}-\mathrm{p}K_{a1}} + 10^{2\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}} + 10^{3\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}-\mathrm{p}K_{a3}}\right)^{-1} \\
\left[\mathrm{H_2A^-}\right] &= [\mathrm{H_3A}] \cdot 10^{\mathrm{pH}-\mathrm{p}K_{a1}} \\
\left[\mathrm{HA^{2-}}\right] &= [\mathrm{H_3A}] \cdot 10^{2\mathrm{pH}-\mathrm{p}K_{a1}-\mathrm{p}K_{a2}} \\
\left[\mathrm{A^{3-}}\right] &= \text{total } [\mathrm{A}] - [\mathrm{H_3A}] - \left[\mathrm{H_2A^-}\right] - \left[\mathrm{HA^{2-}}\right]
\end{aligned}$$

where once again the dilution can be taken into account by modifying the total [A] with the dilution factor.